

Energieoptimierung des LoRa-Sensorknotens

Dokumentation der Maßnahmen und ihrer messtechnischen Verifikation

Hardware-Stack: Adafruit Feather M0 (SAMD21G18A) · RFM95 LoRa (SX1276, 868 MHz, RadioHead) · MCP9600 Thermoelement-Verstärker (Typ K) · TPL5111 Hardware-Power-Timer

Messgerät: Rohde & Schwarz RTB2004 (C1, 10:1-Tastkopf, DC-Kopplung)

1 Grundgedanke der Optimierung

Der Knoten arbeitet nach dem Muster **Wake** → **Measure** → **Transmit** → **Shutdown**. Den langen Ruheintervall zwischen zwei Messungen übernimmt nicht der Mikrocontroller per Software-Sleep, sondern der **TPL5111**, der die Versorgung physisch abtrennt. Dadurch ist der Ruhestrom bereits am physikalischen Optimum:

Ruhekonzept	typ. Ruhestrom	Verhältnis
<code>delay()</code> (MCU läuft durch)	14–16 mA	—
Software-Deep-Sleep (SAMD21, STANDBY)	0.75–2.7 mA	Faktor ~10 besser
TPL5111 (Versorgung abgetrennt)	~35 nA	Faktor ~20.000–30.000 besser

Ein abgeschalteter Chip zieht prinzipbedingt weniger als ein schlafender. **Damit ist die lange Pause bereits gelöst – es gibt dort nichts zu verbessern.** Das gesamte verbleibende Einsparpotenzial liegt deshalb in der **Verkürzung und Entschärfung der aktiven Phase**: Je kürzer der Knoten wach ist und je geringer sein Strom in dieser Zeit, desto weniger Energie pro Zyklus.

Die folgenden Maßnahmen greifen genau dort an. Sie lassen sich in drei Gruppen einteilen:

1. **Kaltstart-Overhead eliminieren** (Bootloader, Quarz-Anschwingzeit)
 - die größten Zeitposten.

2. **Mess-Wartefenster verkürzen** (reduzierte ADC-Auflösung des MCP9600).
3. **Sendephase optimieren** (Payload, Sendeleistung, Transceiver-Sleep).

2 Messmethodik

Die Bewertung jeder Maßnahme erfolgte **empirisch am Oszilloskop**, nicht nur rechnerisch. Beobachtet wurde der Spannungsverlauf an einem Pufferkondensator während eines vollständigen Wake-Zyklus.

Physikalische Grundlage:

$$I = C \cdot \frac{dV}{dt}$$

Die **Steigung der Entladekurve kodiert direkt den Momentanstrom**: Eine steile Flanke bedeutet hohen Strom, eine flache Flanke geringen Strom. So lassen sich die Phasen eines Zyklus optisch trennen:

- **Flache, lange Flanke** → MCU im Startup mit Funk im Idle (geringer, konstanter Strom).
- **Mittelsteile Flanke** → MCU aktiv, MCP9600 konvertiert (Mess-/Wartefenster).
- **Steile, kurze Flanke** → LoRa-PA-Sendepuls (höchster Strom des Zyklus).
- **Erholung der Spannung** → TPL5111 hat die Last abgeschaltet, Quelle entspannt sich.

Ergänzend wurde der Code mit **millis()-Markierungen** im Payload instrumentiert. Damit ließ sich eindeutig nachweisen, ab welcher Flanke der eigene Anwendercode überhaupt läuft – entscheidend für die Diagnose des Kaltstart-Overheads (siehe 3.1/3.2).

3 Die Maßnahmen im Einzelnen

3.1 Bootloader entfernen (Bare-Metal-Flashing)

Problem. Weil der TPL5111 die Versorgung bei jedem Zyklus vollständig trennt, durchläuft der SAMD21 bei *jeder* Messung einen kompletten Cold-Boot. Mit Bootloader läuft vor dem eigentlichen Anwendercode jedes Mal der UF2-Bootloader (Wartefenster, USB-Stack). Dieser Overhead fällt nicht

einmalig, sondern **bei jedem Aufwachen** an und verlängert die aktive Phase.

Lösung. Die Firmware wird **bare-metal ohne Bootloader** direkt ab Adresse `0x00000000` geflasht. Das Programmieren erfolgt über **SWD via OpenOCD**, wobei der Raspberry Pi 5 selbst als SWD-Programmer dient (bitgebangtes SWD über GPIO, `linuxgpiod`-Interface auf `gpiochip0`: `SWDIO/SWDCLK/RESET` an GPIO 24/25/18).

Notwendige Bausteine:

- **Linker-Skript** `flash_without_bootloader.ld` – verlegt FLASH auf `0x00000000` (kein `0x2000`-Offset für den Bootloader). Bestätigt über die Memory-Map: `FLASH 0x00000000 0x00040000 xr`.
- **OpenOCD-Flash-Sequenz:** Bootloader-Schutz auf 0 setzen → `chip-erase` → `program ...verify reset 0x00000000`.

Damit liegt der Reset-Vektor bei `0x0000`, der `Reset_Handler` startet sofort, ohne Bootloader-Vorlauf.

Wirkung. Der Bootloader-Anteil der aktiven Phase entfällt vollständig. Sichtbar im Vergleich der Aufnahmen *mitBootloader* → *ohneBootloader* (siehe Abschnitt 4).

3.2 Quarz-Oszillator abschalten (-DCRYSTALLESS)

Problem. Nach dem Entfernen des Bootloaders blieb am Oszilloskop weiterhin eine **flache Flanke von ~2s** vor dem eigentlichen Stromanstieg sichtbar. Die `millis()`-Instrumentierung bewies: Während dieser Flanke läuft *noch kein* Anwendercode – die Zeit liegt also **vor setup()**, im Core-Startup.

Ursache ist der **XOSC32K**: In `SystemInit()` (Adafruit-SAMD-Core, `cores/arduino/startup.c`) wird der externe 32.768 kHz-Uhrenquarz als Referenz für den DFLL48M aktiviert, und der Code wartet **blockierend** auf das Ready-Flag:

```
SYSCTRL->XOSC32K.bit.ENABLE = 1;
while ((SYSCTRL->PCLKSR.reg & SYSCTRL_PCLKSR_XOSC32KRDY) == 0) { }
// blockiert
```

32 kHz-Uhrenquarze haben prinzipbedingt sehr lange Anschwingzeiten (mehrere hundert ms bis ~2 s). Da der TPL5111 die Versorgung trennt, muss der Quarz bei *jedem* Wake kalt anschwingen – die ~2 s fallen also pro Zyklus an.

Lösung. Kompilieren mit dem Flag `-DCRYSTALLESS`. Der Adafruit-Core ist in `startup.c` über `#if defined(CRYSTALLESS)` zweigeteilt: Im Crystalless-Zweig wird statt des Quarzes der **interne Oszillator (OSC32K)** als DFLL-

Referenz genutzt, der in wenigen Millisekunden bereit ist. Die lange Wait-Schleife entfällt komplett.

Stolperstein beim Build (dokumentiert). `arduino-cli` setzt `build.extra_flags` pro Board in `boards.txt`. Wird die Property nur mit `-DCRYSTALLESS` überschrieben, gehen die essenziellen Device-Defines verloren und der Build schlägt fehl. Das Flag muss daher an den **vollständigen** Original-String **angehängt** werden. Der etablierte Compile-Befehl:

```
arduino-cli compile --fqbn adafruit:samd:adafruit_feather_m0 \
--build-property "build.ldscript=linker_scripts/gcc/
flash_without_bootloader.ld" \
--build-property "build.extra_flags=-D__SAMD21G18A__ -
DADAFRUIT_FEATHER_M0 -DARDUINO_SAMD_ZERO -DARM_MATH_CM0PLUS
{build.usb_flags} -DCRYSTALLESS" \
--output-dir ./build ..
```

(Beim Verifizieren per `grep` ist `--clean` nötig, sonst unterdrückt der Build-Cache die Recompile-Ausgabe.)

Wirkung. Die flache ~ 2 s-Flanke verschwindet vollständig – am Oszilloskop verifiziert. Nach dem Einschalten durch den TPL5111 beginnt der Stromfluss quasi sofort.

Tradeoff. Der interne Oszillator ist ungenauer als der Quarz ($\sim \pm 0.25\%$ bei Raumtemperatur, über Temperatur/Spannung grob ± 1 – 2% statt $\sim \pm 20$ – 50 ppm beim Quarz). Für diesen Knoten **unkritisch**, weil an nichts harter Zeitgenauigkeit hängt: Das LoRa-HF-Timing erzeugt der RFM95 aus seinem *eigenen* Quarz; `delay()/millis()`-Abweichungen von ein paar ms auf eine ~ 200 ms-Aktivzeit sind irrelevant; das TPL5111-Intervall stammt vom externen Widerstand am TPL5111. Relevant würde die Drift nur bei USB-CDC, schnellem UART oder absoluten RTC-Zeitstempeln – alles hier nicht genutzt. Der Quarz bleibt physisch auf dem Board und kann durch Weglassen des Flags jederzeit wieder verwendet werden – `CRYSTALLESS` ist reine Software-Konfiguration.

3.3 ADC-Auflösung des MCP9600 reduzieren

Problem. Die Konversionszeit des MCP9600 hängt direkt von der eingestellten ADC-Auflösung ab. Bei 18 Bit (Standard) dauert eine Konversion ~ 320 ms – während dieser Zeit muss der MCU per Wartefenster (`delay`) wach bleiben. Das war der **größte einzelne Zeitposten der aktiven Phase**.

Datenblatt-Grundlage (Microchip DS20005426, Tabelle 5-3):

(Zur t_{CONV} kommt jeweils die Temperature Calculation Time t_{CALC} von typ. ~ 12 ms hinzu.)

ADC-Auflösung	Konversionszeit (typ.)	Temp.-Auflösung (Typ K)
18 Bit	~320 ms	~0.0625 °C
16 Bit	~80 ms	~0.25 °C
14 Bit	~20 ms	~1 °C
12 Bit	~5 ms	~4 °C

Lösung. Auflösung über `setADCresolution()` herabsetzen und das Wartefenster entsprechend kürzen. Für eine Temperaturdifferenzmessung ist **16 Bit der gute Kompromiss** (~0.25 °C, ~80 ms) – drastisch kürzer als 18 Bit, ohne die Genauigkeit auf nutzlose ~1 °C (14 Bit) zu reduzieren. Das Wartefenster muss dann über $t_{\text{CONV}} + t_{\text{CALC}}$ liegen (z. B. `delay(100)` bei 16 Bit; bei 14 Bit reichen ~50 ms).

Wichtige Einschränkung. `setADCresolution()` betrifft **nur** die Heistellen- (Hot-Junction-)Konversion. Der Kaltstellensensor (Ambient) hat eine **separate** Auflösungseinstellung (`setAmbientResolution()`, Standard ~0.25 °C). Auerdem: Bei 14 Bit springt die Heistellentemperatur in ~1 °C-Schritten – ein `%.1f`-Format tuscht dann eine Zehntelgrad-Genauigkeit vor, die der Sensor in dieser Stufe nicht liefert.

Wirkung. Verkrzung des Mess-Wartefensters von ~320 ms auf ~80–100 ms (16 Bit) bzw. ~50 ms (14 Bit) – die mittelsteile Flanke der aktiven Phase wird deutlich schmaler.

3.4 Sendephase optimieren

Der LoRa-Sendepuls ist der **hchste Strom des gesamten Zyklus** (bei 10 dBm/PA_BOOST typ. ~80–100 mA fr die Dauer der Airtime). Drei Hebel:

a) Kompaktes Binr-Payload statt CSV-String. Statt eines CSV-Strings (~17 Byte) werden die Werte als `int16_t` in Zehntelgrad skaliert bertragen (`__attribute__((packed))` gegen Padding). Da von den drei Werten (t_h , t_c , Δt) jeder zwei den dritten bestimmt, ist Δt mathematisch redundant und kann empfngerseitig berechnet werden → 4 Byte statt 6. Bei SF7/125 kHz/CR4/5 reduziert das die Airtime von ~51 ms auf ~31 ms (~40 % weniger TX-Energie).

Grenze beachten: Bei SF7/BW125/CR4/5 fallen Payloads von 1–5 Byte in denselben Symbolblock (18 Symbole). Eine weitere Verkleinerung *unterhalb* dieser Grenze bringt dann keinen zustzlichen Airtime-Gewinn mehr – der dominierende Anteil ist dann

die Präambel ($\sim 40\%$ der ToA). Der Sprung von CSV (~ 17 Byte) auf das kompakte Binärformat ist der eigentliche Hebel.

b) Sendeleistung an die Strecke anpassen. `setTxPower(10, false)` – 10 dBm statt 17/20 dBm. Jedes dB weniger reduziert den TX-Strom merklich; bei kurzer Funkstrecke vertretbar.

c) RFM95 nach dem Senden explizit schlafen legen. `rf95.sleep()` (Register `0x01` $\rightarrow$ Mode = Sleep) nach dem TX. Viele Libs lassen den RFM95 sonst im STANDBY (~ 1.6 mA) – messbar und vermeidbar. Im Sleep zieht der Transceiver nur ~ 0.2 μ A.

4 Wirkung im Oszilloskop – die Optimierungsstufen im Vergleich

Die vier Aufnahmen dokumentieren die schrittweise Verkürzung der aktiven Phase. Alle zeigen denselben Zyklus (Mess- und Sendephase + TX-Drop), aber in verschiedenen Optimierungsstufen. Maßgeblich ist die **zeitliche Breite der aktiven Phase** vor der Spannungserholung.

Lesart der Bilder.

Abb. 1 (mitBootloader) zeigt die längste, flache Anlaufphase: Bootloader-Vorlauf *und* Quarz-Anschwingen liegen vor dem eigentlichen Code. Die lineare Spannungsabnahme ist der MCU bei ~ 48 MHz mit Funk im Idle.

Abb. 2 (ohneBootloader) belegt, dass nach dem Bare-Metal-Flashing noch eine ~ 2 s-flache Flanke bleibt – das ist isoliert die **XOSC32K-Anschwingzeit**. Genau diese Flanke war der Auslöser für Maßnahme 3.2.

Abb. 3 (ohneBootloader+ohneClock) ist der angestrebte Endzustand: Die 2 s-Flanke ist weg, die gesamte aktive Phase passt in ~ 300 ms; deutlich erkennbar nur noch die kurze Mess-/Wartephase (mittelsteil) und der steile TX-Drop am Cursor.

Abb. 4 (reduzierte Auflösung) illustriert die Wirkung der verkürzten Sensor-Wartezeit: Die mittelsteile Mess-Flanke ist auf wenige hundert ms zusammengeschrumpft, gefolgt vom scharfen TX-Einbruch und der sofortigen Erholung (TPL5111 schaltet ab).

Hinweis: Die Cursor- Δt -Werte messen je nach Bild leicht unterschiedliche Abschnitte (Zeitbasis variiert). Aussagekräftig ist der **qualitative Vergleich der Phasenbreiten**, nicht der direkte Zahlenvergleich zwischen Bildern mit unterschiedlicher Zeitbasis.

#	Stufe	Zeitbasis	Δt (Cursor)	Zustand
1	mit Bootloader	400 ms/div	2.316 s	Ausgangszustand: mit Bootloader + Quarz
2	ohne Bootloader	480 ms/div	2.016 s	Bootloader entfernt – aber noch mit Quarz: die flache ~2 s-XOSC32K- Flanke ist klar sichtbar
3	ohne Bootloader + ohne Clock	150 ms/div	307.5 ms	Endzustand: zusätzlich -DCRYSTALLESS – die 2 s-Flanke ist verschwunden
4	reduzierte Auflösung	500 ms/div	185 ms	Illustration der verkürzten Mess- /Wartephase (reduzierte ADC- Auflösung, entferntes 100-ms-Fenster)

5 Zusammenfassung

Kernaussage: Weil das Power-Gating den Ruhestrom bereits an die physikalische Untergrenze bringt, bestand der gesamte Optimierungsweg darin, die **aktive Phase systematisch zu verkürzen** – vom Kaltstart-Overhead (Bootloader, Quarz) über das Mess-Wartefenster (reduzierte ADC-Auflösung) bis zur Sendephase (Payload, Leistung). Jeder Schritt wurde am Oszilloskop über das Prinzip $I = C \cdot dV/dt$ verifiziert; die vier Aufnahmen belegen die Verkürzung von über 2 s Anlaufzeit auf eine aktive Gesamtphase im Bereich weniger hundert Millisekunden.

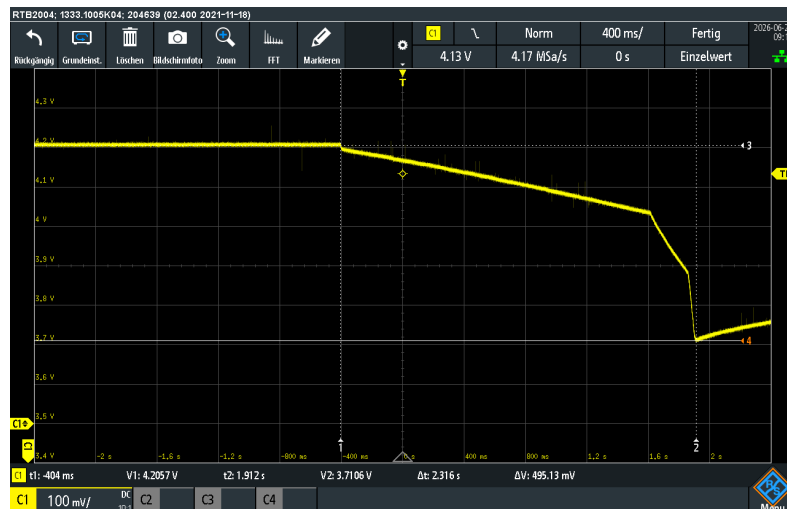


Abbildung 1: Stufe 1 – mit Bootloader und Quarz (Ausgangszustand). Lange flache Anlaufphase aus Bootloader-Vorlauf und Quarz-Anschwingen.



Abbildung 2: Stufe 2 – ohne Bootloader, aber noch mit Quarz. Die isolierte ~ 2 s-XOSC32K-Anschwingflanke bleibt sichtbar.

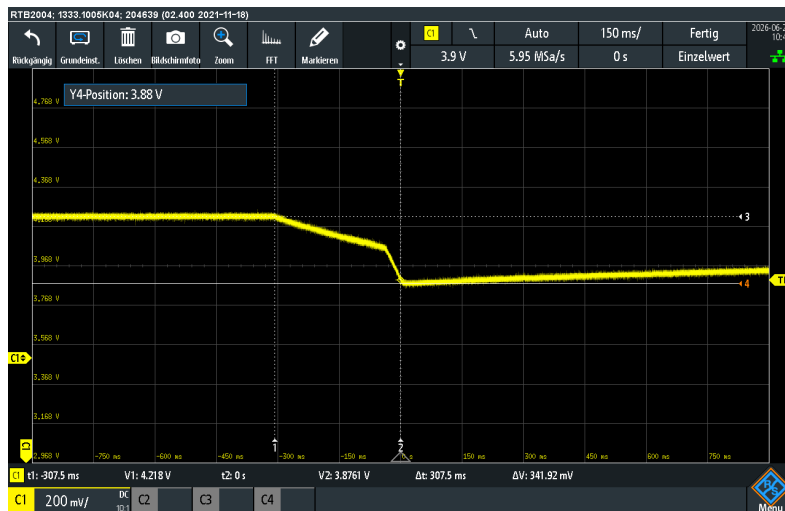


Abbildung 3: Stufe 3 – ohne Bootloader und mit -DCRYSTALLESS (Endzustand). Die 2 s-Flanke ist verschwunden; die aktive Phase passt in ~ 300 ms.

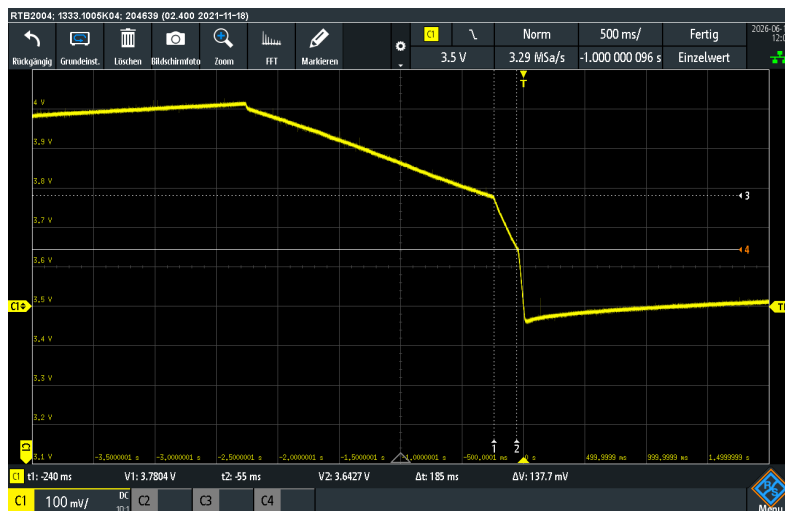


Abbildung 4: Reduzierte ADC-Auflösung mit entferntem 100-ms-Fenster: die Mess-/Wartephase ist auf wenige hundert Millisekunden verkürzt.

Maßnahme	Angriffspunkt	Mechanismus	Ergebnis
TPL5111-Power-Gating	Ruhephase	Versorgung physisch trennen	~35 nA statt mA-Bereich (Faktor ~20.000)
Bootloader entfernen	Kaltstart	Bare-Metal-Flash ab 0x0000 via SWD	Bootloader-Vorlauf pro Zyklus eliminiert
-DCRYSTALLESS	Kaltstart	interner OSC statt XOSC32K	~2 s pro Zyklus eingespart
ADC-Auflösung ↓	Mess-Wartefenster	kürzere Konversionszeit	~320 ms → ~80/50 ms
Binär-Payload	Sendephase	int16 ×10 statt CSV	Airtime ~51 ms → ~31 ms
setTxPower(10)	Sendephase	reduzierte PA-Leistung	geringerer TX-Strom
rf95.sleep() nach TX	nach Sendephase	Transceiver in Sleep	~0.2 µA statt ~1.6 mA